

HTA PLATFORM USER MANUAL

Version 1.0

Installation, Configuration and User Guide (Windows & macOS)

Document Control

Document Version	1.0
Application	HTA Platform
Prepared By	Shantanu Fulaware
Audience	Non-Technical Users
Supported OS	Windows 10/11 and macOS

Document Revision History

Version	Date	Description
1.0	Initial	Basic User Manual

Table of Contents

1. Introduction
2. System Requirements
3. Software Installation
4. Installing the HTA Platform
5. Running the HTA Platform
6. Using the HTA Platform
7. Project Folder Structure
8. Troubleshooting
9. Frequently Asked Questions
10. Appendix

1. Introduction

1.1 Purpose of this Manual

This manual provides step-by-step instructions for installing, configuring and using the HTA Platform. It is written for users with little or no programming experience and includes instructions for both Windows and macOS.

1.2 About the HTA Platform

The HTA Platform is a Python and Streamlit-based application that supports Health Technology Assessment data extraction workflows. It combines web scraping, PDF table extraction and Retrieval-Augmented Generation (RAG) document search into a single application.

1.3 Main Features

- NICE website web scraping
- PDF table extraction to Excel
- AI-powered document question answering using Ollama
- Automatic vector database creation
- Excel report generation
- User-friendly Streamlit interface
- Runs locally without requiring cloud services

1.4 Intended Users

The HTA Platform is intended for Health Technology Assessment analysts, researchers, healthcare consultants, students and academic users. No prior programming knowledge is required.

2. System Requirements

Component	Windows	macOS
Operating System	Windows 10/11	Ventura or later
Python	3.11	3.11
Visual Studio Code	Required	Required
Ollama	Required	Required
Ghostscript	Required	Required
RAM	8 GB minimum	8 GB minimum
Free Disk Space	10 GB	10 GB

Important Note

Use Python 3.11 for this project. Newer Python versions may not be compatible with some AI libraries used by the application.

Before You Begin

- ☐ Administrator access to your computer
- ☐ Visual Studio Code installed
- ☐ Python 3.11 installed
- ☐ Stable internet connection
- ☐ Project files copied to your computer
- ☐ Approximately 10 GB of free disk space

End of Part 1

The next section of the manual covers software installation, project setup, creating the virtual environment and installing dependencies.

HTA Platform User Manual - Version 1.0

Part 2 - Software Installation

This section explains how to install all software required before using the HTA Platform. Follow the steps appropriate for your operating system.

2.1 Install Visual Studio Code

1. Open your web browser.
2. Download Visual Studio Code from <https://code.visualstudio.com>.
3. Run the installer.
4. Accept the default settings.
5. Start Visual Studio Code after installation. Expected Result: VS Code opens successfully.

2.2 Install Python 3.11

Windows:

1. Download Python 3.11 from <https://www.python.org/downloads/>.
2. During installation, tick 'Add Python to PATH'.
3. Finish the installation.

macOS:

1. Download Python 3.11 from python.org.
2. Install the package.
3. Verify installation from Terminal:
`python3.11 --version`

Expected Result: Python 3.11.x is displayed.

2.3 Install Homebrew (macOS Only)

Skip this section if you are using Windows.

1. Open Terminal.
2. Install Homebrew using the official installation command from <https://brew.sh>.
3. Verify installation:
`brew --version`

Expected Result: Homebrew version number is displayed.

2.4 Install Ghostscript

Windows:

- Download and install Ghostscript from <https://ghostscript.com/releases/>.
- Accept the default installation settings.

macOS:

- Open Terminal.
- Run:
`brew install ghostscript`

Verify installation:

```
gs --version
```

Expected Result: A Ghostscript version number is displayed.

2.5 Install Ollama

1. Download Ollama from <https://ollama.com/download>.
2. Install the application.
3. Start Ollama.
4. Verify installation:
`ollama --version`

If this is your first installation, download the language model: `ollama pull llama3.1:8b`

To check installed models:

```
ollama list
```

2.6 Verify All Software

Before proceeding, verify the following:

- ✓ Visual Studio Code opens successfully.
- ✓ Python 3.11 is installed.
- ✓ Ghostscript is installed.
- ✓ Ollama is installed.
- ✓ Internet connection is available.

If any verification fails, resolve the issue before continuing to Part 3.

Important Notes

- Always use Python 3.11 for this project.
- Install Ghostscript before using the PDF Table Extraction module.
- Ollama must be running before using the RAG Chat module.
- Do not continue until all software has been successfully installed.

Next Section

Part 3 explains how to copy the HTA Platform project, open it in Visual Studio Code, create the virtual environment, install all project dependencies and verify the installation.

HTA Platform User Manual - Version 1.0

Part 3 – Project Installation and Environment Setup

This section explains how to copy the HTA Platform project, open it in Visual Studio Code, create the Python virtual environment and install all required project dependencies.

3.1 Copy the Project Files

1. Create a folder named 'NICE HTA PLATFORM' on your Desktop.
2. Copy all project files into this folder.
3. Ensure files such as `streamlit_app.py`, `requirements-vscode.txt`, `environment-vscode.yml` and the `ui` folder are present.
4. Do NOT manually create folders such as `assets`, `config`, `documents`, `outputs`, `tables`, `temp` or `vectorstore`. These are created automatically by the application.

3.2 Open the Project in Visual Studio Code

1. Open Visual Studio Code.
2. Select File → Open Folder.
3. Browse to 'NICE HTA PLATFORM'.
4. Click Open.
5. Open the integrated terminal using Terminal → New Terminal.

Expected Result:

The project files appear in the Explorer panel and a terminal opens at the project location.

3.3 Create a Virtual Environment

Windows

```
python -m venv venv
```

macOS

```
python3.11 -m venv venv
```

Expected Result: A new folder named 'venv' is created inside the project folder.

3.4 Activate the Virtual Environment

Windows

```
venv\Scripts\activate
```

macOS

```
source venv/bin/activate
```

Expected Result: The terminal prompt starts with (venv).

3.5 Verify the Python Version

Run the following command:

```
python --version
```

Expected Result: Python 3.11.x

Important: Do not continue if Python 3.14 or another version is displayed.

3.6 Upgrade pip

Run:

```
python -m pip install --upgrade pip
```

3.7 Install Project Dependencies

Run:

```
pip install -r requirements-vscode.txt
```

Note: The installation may take 10–30 minutes depending on your internet connection because AI libraries such as PyTorch and Transformers are large downloads.

3.8 Verify the Installation

Verify the following commands complete successfully:

- `python --version`
- `pip list`
- `gs --version`
- `ollama --version`

Common Installation Issues

Issue	Solution
-------	----------

Python version is not 3.11	Recreate the virtual environment using Python 3.11.
ModuleNotFoundError	Ensure the virtual environment is activated before running the application.
Ghostscript not found	Install Ghostscript and verify using 'gs --version'.
Installation takes a long time	This is normal while downloading AI packages such as PyTorch.

Next Section

Part 4 explains how to start Ollama, launch the Streamlit application, access the HTA Platform in your browser, and correctly start the application each day.

HTA Platform User Manual - Version 1.0

Part 4 – Starting and Running the HTA Platform

This section explains the daily startup procedure, how to start Ollama, launch the Streamlit application, access the HTA Platform in your browser, and safely close the application.

4.1 Daily Startup Checklist

1. Turn on your computer and connect to the internet.
2. Open Visual Studio Code.
3. Open the 'NICE HTA PLATFORM' project folder.
4. Open a new terminal in VS Code.
5. Activate the virtual environment.
6. Start Ollama.
7. Run the Streamlit application.
8. Open the application in your web browser.

4.2 Activate the Virtual Environment

Windows

```
venv\Scripts\activate
```

macOS

```
source venv/bin/activate
```

Expected Result: The terminal prompt begins with (venv).

4.3 Start Ollama

If using the Ollama desktop application, open it and leave it running. Alternatively, start the

Ollama server from the terminal:

```
ollama serve
```

Verify the installed models:

```
ollama list
```

If no model is available, download one:

```
ollama pull llama3.1:8b
```

4.4 Start the Streamlit Application

Always start the application using the following command:

```
python -m streamlit run streamlit_app.py
```

Important: Use the command above instead of 'streamlit run streamlit_app.py'. This ensures Streamlit uses the Python interpreter inside the project's virtual environment.

4.5 Verify Successful Startup

A successful startup displays output similar to:

```
Local URL: http://localhost:8501  
Network URL: http://<your-ip-address>:8501
```

Open `http://localhost:8501` in your web browser if it does not open automatically.

4.6 Main Application Screen

After startup, the HTA Platform home page is displayed. The application provides access to:

- Web Scraper
- Table Extraction
- RAG Chat

4.7 Daily Shutdown Procedure

9. Close the browser tab.
10. Return to the VS Code terminal.
11. Press Ctrl + C to stop the Streamlit server.
12. Type 'deactivate' to exit the virtual environment.
13. Close Visual Studio Code.
14. Close Ollama if it is no longer required.

4.8 Common Startup Problems

Problem	Solution
Application does not open	Ensure Ollama is running and the virtual environment is activated.
No module named 'bs4'	Run the application using 'python -m streamlit run streamlit_app.py'.
Ghostscript warning	Install Ghostscript and verify with 'gs --version'.
Port 8501 already in use	Stop the previous Streamlit session or restart the terminal.
Ollama connection error	Start Ollama using 'ollama serve' or launch the Ollama desktop application.

Next Section

Part 5 explains how to use each HTA Platform module, including the Web Scraper, PDF Table Extraction and RAG Chat, with step-by-step examples.

HTA Platform User Manual - Version 1.0

Part 5 – Using the HTA Platform

This section explains how to use each module of the HTA Platform. Follow the steps in the order presented. Screenshot placeholders have been included so they can be replaced with screenshots from the final application.

5.1 HTA Platform Home Screen

After starting the application, the home page displays the available modules.

Available modules:

- Web Scraper
- PDF Table Extraction
- RAG Chat

NOTE : ALWAYS ENTER TA NUMBER AS – TA920 or TA130 – don't give space between TA and number

5.2 Module 1 – Web Scraper

Purpose

The Web Scraper extracts publicly available information from NICE web pages and stores the extracted information for further analysis.

Steps

1. Select the Web Scraper module.
2. Enter the NICE Technology Appraisal URL (if required).
3. Click the 'Start Scraping' button.
4. Wait for the scraping process to complete.
5. Review the extracted information.
6. Save or export the results if required. Expected

Output

Relevant NICE information is displayed and can be exported for further analysis.

5.3 Module 2 – PDF Table Extraction

Purpose

This module extracts tables from PDF reports and converts them into Microsoft Excel format.

Steps

7. Open the Table Extraction module.
8. Click 'Browse' and select the PDF file.
9. Click 'Extract Tables'.
10. Wait until processing is complete.
11. Review the extracted tables.
12. Download the generated Excel file. Expected

Output

Tables are extracted and saved as an Excel workbook.

Ghostscript must be installed before using this module.

5.4 Module 3 – RAG Chat

Purpose

The RAG Chat module allows users to upload documents, create a searchable knowledge base and ask questions in natural language.

Steps

13. Open the RAG Chat module.
14. Upload one or more supported documents.
15. Click 'Build Vector Database' (or equivalent button).
16. Wait until indexing is complete.
17. Enter your question in the chat box.
18. Press Enter or click 'Ask'.
19. Review the AI-generated response. Expected

Output

The system searches the uploaded documents and returns an AI-generated answer based on the indexed content.

5.5 Exporting Results

Depending on the module used, outputs may include:

- Excel workbooks
- Extracted tables
- Scraped data
- AI-generated responses

These files are saved in the project output folders created automatically by the application.

5.6 Good Practice

- Keep Ollama running while using the RAG Chat module.
- Do not close the terminal while the application is running.
- Wait for long-running tasks such as PDF extraction to complete.
- Store project files only inside the NICE HTA PLATFORM folder.
- Regularly back up exported Excel files and reports.

Next Section

Part 6 explains the project folder structure, describes automatically created folders and identifies where uploaded documents and generated outputs are stored.

HTA Platform User Manual - Version 1.0

Part 6 – Project Folder Structure and Data Management

This section explains the purpose of each project file and folder, identifies which folders are created automatically by the application, and describes where input and output files are stored.

6.1 Project Folder Structure

Example Project Structure

NICE HTA PLATFORM

```
|
|— streamlit_app.py
|— rag.py
|— ta_scraper.py
|— excel_manager.py
|— excel_formatter.py
|— hta_table_engine.py
|— repository.py
|— table_repository.py
|— paths.py
|— requirements-vscode.txt
|— environment-vscode.yml
|— ui/
|— venv/
```

6.2 Main Project Files

File / Folder	Purpose
streamlit_app.py	Main application entry point that launches the HTA Platform.
ui/	Contains the Streamlit user interface modules (Web Scraper, Table Extraction and RAG Chat).
rag.py	Implements Retrieval-Augmented Generation (RAG) functionality.
ta_scraper.py	Retrieves information from NICE web pages.
hta_table_engine.py	Processes PDF documents and extracts tables.
excel_manager.py	Creates and manages Excel workbooks.
excel_formatter.py	Applies formatting to generated Excel files.

repository.py	Manages the document repository.
table_repository.py	Stores and manages extracted table information.
paths.py	Defines and manages project file paths.
requirements-vscode.txt	List of Python packages required by the application.
environment-vscode.yml	Optional Conda environment definition.
venv/	Python virtual environment containing installed packages.

6.3 Automatically Created Folders

The following folders are created automatically by the application when required. Users do not need to create these folders manually.

Folder	Created By	Purpose
assets	Application	Stores application assets and supporting files.
config	Application	Stores configuration files.
documents	Application	Stores uploaded source documents.
outputs	Application	Stores generated reports and exported files.
tables	Application	Stores extracted table data.
temp	Application	Stores temporary processing files.
vectorstore	Application	Stores the vector database used by the RAG module.

6.4 Data Flow

1. The user uploads documents or starts web scraping.
2. The application processes the data.
3. Intermediate files are stored in temporary folders.
4. Final outputs are saved in the outputs folder.
5. Documents indexed for AI search are stored in the vectorstore.

6.5 Backup Recommendations

- Back up the entire 'NICE HTA PLATFORM' folder regularly.
- Do not delete the 'venv' folder unless recreating the environment.
- Do not modify application source files unless instructed by a developer.
- Back up the outputs folder if reports or Excel files are important.
- Keep uploaded source documents in the documents folder.

6.6 Important Notes

- Do not rename project files unless instructed.
- Do not manually create the automatically generated folders.

- If an automatically generated folder is accidentally deleted, it will normally be recreated the next time the application runs.

Next Section

Part 7 covers troubleshooting, frequently asked questions and solutions to the most common installation and runtime issues.

HTA Platform User Manual - Version 1.0

Part 7 – Troubleshooting, Frequently Asked Questions and Support

This section provides solutions to the most common installation and runtime issues encountered when using the HTA Platform.

7.1 Troubleshooting Guide

Problem	Possible Cause	Recommended Solution
Python version is not 3.11	Incorrect Python installation	Install Python 3.11 and recreate the virtual environment.
ModuleNotFoundError	Virtual environment not activated	Activate the virtual environment before running the application.
No module named 'bs4'	Streamlit using the wrong Python interpreter	Run the application using 'python -m streamlit run streamlit_app.py'.
Ghostscript is not installed	Ghostscript missing	Install Ghostscript and verify using 'gs --version'.
Cannot connect to Ollama	Ollama server not running	Launch the Ollama application or run 'ollama serve'.
No AI model available	Language model not downloaded	Run 'ollama pull llama3.1:8b'.
Application does not open	Streamlit failed to start	Check the terminal for error messages and restart the application.
Port 8501 already in use	Another Streamlit instance is running	Stop the previous Streamlit process or restart the terminal.
PDF table extraction fails	Unsupported or scanned PDF	Check the PDF quality and ensure Ghostscript is installed.
RAG search returns no answer	Documents not indexed	Upload documents and rebuild the vector database.

7.2 Frequently Asked Questions (FAQ)

Q: Can I use Python 3.14?

A: No. This project is designed and tested with Python 3.11.

Q: Do I need an internet connection?

A: An internet connection is required during installation. Daily use depends on your workflow; local AI features run through Ollama.

Q: Do I need to recreate the virtual environment every time?

A: No. Create it once and activate it whenever you use the project.

Q: Should I create the assets, outputs or vectorstore folders manually?

A: No. The application creates these folders automatically when needed.

Q: Why do I use 'python -m streamlit' instead of 'streamlit run'?

A: It ensures Streamlit uses the Python interpreter from the project's virtual environment.

Q: Can I move the project folder?

A: Yes. After moving it, open the new location in VS Code and reactivate the virtual environment if required.

7.3 Daily Best Practices

- Keep Ollama running while using the RAG Chat module.
- Always activate the virtual environment before launching the application.
- Close the application using Ctrl+C before shutting down the computer.
- Regularly back up the project and exported reports.
- Install updates only after creating a backup.

7.4 Support Checklist

Before requesting technical support, collect the following information:

- Operating system (Windows or macOS)
- Python version
- Ghostscript version
- Ollama version
- Screenshot of the error message
- Terminal output (if available)
- Description of the task being performed

7.5 Final Pre-Use Checklist

- ☐ VS Code opens correctly.
- ☐ Virtual environment is activated.
- ☐ Python 3.11 is in use.
- ☐ Ghostscript is installed.
- ☐ Ollama is running.
- ☐ Required AI model is available.
- ☐ Application starts successfully using 'python -m streamlit run streamlit_app.py'.

Next Section

Part 8 (Final Appendix) contains a Quick Start Guide, Daily Startup Reference, commonly used commands for Windows and macOS, maintenance recommendations and a final deployment checklist.

HTA Platform User Manual - Version 1.0

Part 8 – Appendix, Quick Reference and Maintenance

8.1 Quick Start Guide

Follow these steps each time you want to use the HTA Platform:

1. Open Visual Studio Code.
2. Open the 'NICE HTA PLATFORM' project folder.
3. Open a new Terminal window.
4. Activate the virtual environment.
5. Start Ollama.
6. Run the Streamlit application.
7. Open the browser at <http://localhost:8501>.
8. Use the required HTA Platform module.

8.2 Daily Startup Commands

Task	Windows	macOS
Activate virtual environment	venv\Scripts\activate	source venv/bin/activate
Start Ollama	ollama serve	ollama serve
Run application	python -m streamlit run streamlit_app.py	python -m streamlit run streamlit_app.py
Stop application	Ctrl + C	Ctrl + C
Exit virtual environment	deactivate	deactivate

8.3 Routine Maintenance

- Back up the complete project folder regularly.
- Keep Python packages up to date when approved by the project administrator.
- Ensure Ollama is updated periodically.
- Retain exported Excel reports in a secure location.
- Remove temporary files only when the application is closed.

8.4 Updating the Project

1. Back up the project.
2. Replace the application files with the updated version.
3. Activate the virtual environment.
4. Run 'pip install -r requirements-vscode.txt' if dependencies have changed.
5. Restart the application and verify normal operation.

8.5 Final Deployment Checklist

- ☐ VS Code installed
- ☐ Python 3.11 installed
- ☐ Ghostscript installed
- ☐ Ollama installed
- ☐ Virtual environment created
- ☐ Dependencies installed
- ☐ Application launches successfully
- ☐ Web Scraper tested
- ☐ PDF Table Extraction tested
- ☐ RAG Chat tested

8.6 Contact and Support

If you encounter problems that cannot be resolved using this manual, contact the project maintainer. When requesting support, include the operating system, Python version, screenshots of the error, and the terminal output.

8.7 Conclusion

The HTA Platform provides an integrated environment for NICE web scraping, PDF table extraction and AI-assisted document analysis. Following the installation and operating procedures in this manual will help ensure reliable day-to-day operation on both Windows and macOS systems.

End of User Manual

Congratulations! You have completed the HTA Platform User Manual. Keep this document with the project files for future reference and user onboarding.